

CG_HW1_MiniDraw 实验报告

李亭皓 PB23010374

2026 年 3 月 8 日

1 问题背景

本次作业要求基于课程提供的 `Framework2D` 框架，实现一个简化的矢量绘图小工具 `MiniDraw`。根据作业说明，系统至少需要支持直线 (`Line`)、矩形 (`Rectangle`)、椭圆 (`Ellipse`)、多边形 (`Polygon`)、自由划线 (`Freehand`) 等图元，并要求通过“类的继承 + 多态”的方式组织程序结构，而不是把所有绘制逻辑堆叠在同一个函数中。除此之外，作业 PPT 还将线宽、颜色、填充、对象选取、对象变换等功能列为扩展方向。

2 系统设计与算法原理

2.1 基类 `Shape` 的抽象作用

所有图元都继承自 `Shape` 基类。基类统一定义了以下接口：

- `draw(const Config&)`: 绘制图元；
- `update(float x, float y)`: 在交互过程中更新图元；
- `add_control_point(float x, float y)`: 为折线类或点集类图元增加控制点；
- `hit_test(float x, float y)`: 判断鼠标是否命中该图元；
- `draw_selected(const Config&)`: 绘制选中高亮；
- `translate/scale/rotate/get_bbox`: 为后续几何编辑预留统一接口。

同时，`Shape::Config` 中统一保存线条颜色、线宽、是否描边、是否填充、填充颜色以及画布偏移量等绘制样式。这样做有两个好处：

2.2 `Canvas` 的逻辑

`Canvas` 是整个系统的交互核心。程序使用一个不可见按钮覆盖整个画布区域，以此统一捕获鼠标输入；随后根据当前工具类型和鼠标事件，分别触发点击、移动、释放、右键、双击等事件处理函数。

其总体逻辑可概括为：

1. 每一帧先绘制背景与边框；
2. 用不可见按钮接收鼠标事件；
3. 若鼠标位于画布内，则根据事件类型调用相应处理函数；
4. 最后绘制历史图元和当前正在预览的图元。

其中最关键的状态变量是：

- `shape_type_`：当前工具类型；
- `draw_status_`：是否处于“正在创建图元”的中间状态；
- `current_shape_`：当前尚未提交到历史列表的图元对象；
- `shape_list_`：已经完成绘制的图元集合；
- `selected_shape_index_`：当前选中的图元索引。

2.3 功能交互逻辑设计

系统的交互方式如下。

2.3.1 普通两点式图元：Line / Rect / Ellipse / Polygon

对于直线、矩形、椭圆、正多边形这类可由“起点 + 拖动终点”定义的图元，采用统一的两阶段交互流程：

1. 用户先点击左键，记录起点，同时创建一个当前图元对象；
2. 鼠标移动时，持续调用 `update()` 更新终点，实现实时预览；
3. 再次点击左键，将当前图元压入历史列表，完成创建。

之所以采用这一方案，是因为这些图元都可以自然地用一个包围框或两个端点参数化，实时更新成本低，界面行为也与常见绘图软件一致，用户容易理解。

2.3.2 Pen 工具：点击加点，双击结束，右键闭合

Pen 工具的设计目标不是记录鼠标连续轨迹，而是模拟“钢笔/折线”式交互。具体逻辑为：

1. 第一次左键点击时创建图元并加入首个控制点；
2. 鼠标移动时通过 `update()` 修改末尾的预览点，使最后一段边实时跟随鼠标；
3. 再次左键点击时，将预览点固化为新的顶点，并继续添加下一段；
4. 双击左键表示结束当前折线；

5. 右键表示闭合折线，并将其作为闭合图形加入历史列表。

采用这种设计而不是“移动即采样”的原因有两点：

1. 它更符合钢笔工具“逐点确定关键顶点”的使用习惯；
2. 点数更少，几何结构更清晰，后续若要做编辑、闭合、多边形化处理也更方便。

2.3.3 Freehand 工具：按住左键连续采样，松开结束

自由划线工具与 Pen 不同，它强调记录手势轨迹，因此交互逻辑为：

1. 左键按下时创建图元，并记录起始点；
2. 鼠标拖动过程中持续采样，将采样点加入点列；
3. 左键释放时结束绘制，并将图元加入历史列表。

为了避免采样过密导致点数爆炸、绘制开销增大和轨迹抖动，本实现没有对每一帧鼠标位置都直接追加，而是设置了一个最小距离阈值：累计位移超过阈值时才真正写入新点，这比起单纯的与上一个点的距离检测，能有效避免用户缓慢小幅度绘制时，不加入新的点的问题。

2.3.4 Select 工具：高亮显示

选择工具采用“从后往前遍历图元列表”的方式实现命中测试。这样设计的原因在于：后绘制的图元通常位于视觉更上层，也应优先被选中，符合用户直觉。

当命中某个图元后，记录其下标；渲染阶段遍历历史图元时，如果某个下标等于当前选中索引，则调用 `draw_selected()` 进行高亮显示，否则调用普通 `draw()`。

这种设计避免了把“选中状态”硬编码进每个对象内部，而是由 Canvas 统一调度，逻辑更集中。

2.4 各图元的几何建模方法

2.4.1 椭圆

椭圆仍然沿用“起点与终点确定外接矩形”的建模方式。先计算中心点：

$$c_x = \frac{x_s + x_e}{2}, \quad c_y = \frac{y_s + y_e}{2},$$

再计算横纵半轴：

$$r_x = \frac{|x_e - x_s|}{2}, \quad r_y = \frac{|y_e - y_s|}{2}.$$

然后调用 `AddEllipse` 和 `AddEllipseFilled` 完成绘制。这种实现的优点是公式简单、实时更新成本低，同时与矩形共享相同的交互方式。

在命中测试方面，椭圆没有使用简单包围盒判断，而是基于标准椭圆方程

$$\frac{(x - c_x)^2}{r_x^2} + \frac{(y - c_y)^2}{r_y^2}$$

进行判定。对于填充椭圆，只要该值不超过 1 即认为命中；对于非填充椭圆，则构造“外扩椭圆”和“内缩椭圆”之间的环带，只有点落在这一区域内才认为点击到了轮廓。这一做法比纯包围盒命中测试更贴近真实几何边界，也能在大小差异很大的椭圆之间保持较一致的交互体验。

2.4.2 正多边形

多边形并不是任意顶点输入，而是基于“外接椭圆参数化”的正多边形。设中心为 (c_x, c_y) ，横纵方向半径为 (r_x, r_y) ，边数为 n ，则第 i 个顶点由下式给出：

$$x_i = c_x + r_x \cos \theta_i, \quad y_i = c_y + r_y \sin \theta_i,$$

其中

$$\theta_i = \theta_0 + \frac{2\pi i}{n}.$$

这里初始角 θ_0 的选取专门做了区分：

- 奇数边时，让一个顶点朝上；
- 偶数边时，让一条边朝上。

这样做的原因不是数学上的必要，而是视觉上的美观性与用户直觉：三角形、五边形等奇数边图形通常更适合“顶点朝上”的姿态；而四边形、六边形等偶数边图形让一条边水平放置，看起来更稳定、更符合常见图标习惯。

2.4.3 Pen 与 Freehand 的点列表示

Pen 和 Freehand 均采用点列 $(x_list_, y_list_)$ 存储几何信息，但两者语义不同：

- **Pen**：点列表示用户明确点击得到的控制点序列，末尾额外保留一个“预览点”；
- **Freehand**：点列表示连续采样得到的轨迹离散点。

2.5 ImGui 绘图接口的优点与局限

ImGui 的优势在于：上手快、集成轻、立即模式写法直观，适合课程作业原型开发。但它本质上不是专业矢量图形编辑内核，因此存在明显限制。

2.5.1 优点

- 直接提供 `AddLine`、`AddRect`、`AddEllipse`、`AddPolyline` 等接口；

- 和按钮、输入框、颜色选择器等 UI 控件天然集成；
- 不需要额外搭建复杂事件系统，较适合教学使用。

2.5.2 局限

(1) **自交多边形填充处理能力有限** ImGui 中用于填充的接口如 `AddConvexPolyFilled` 假设输入是**凸多边形**。这意味着：

- 对于一般凹多边形，直接填充可能出错；
- 对于自交多边形（如五角星式输入），更无法正确根据奇偶规则或非零环绕规则得到预期填充结果。

因此，如果要真正支持任意用户输入多边形，必须引入更完整的多边形三角剖分算法或扫描线填充算法，而不能仅依赖 ImGui 自带接口。在当前实现中，多边形是规则凸多边形，因此该问题被限制在了可控范围内。

(2) **矩形旋转不能靠更新两个角点解决** ImGui 的 `AddRectFilled(const ImVec2& p_min, const ImVec2& p_max, ...)` 从接口定义上就决定了它描述的是**轴对齐矩形**。因此，若要实现矩形旋转，不能简单地“旋转 startpoint 和 endpoint”。

3 实验结果与分析

3.1 已实现功能总结

1. 直线、矩形、椭圆、规则多边形、钢笔折线、自由划线等图元的创建；
2. 描边开关、线宽调节、描边颜色设置；
3. 填充开关与填充颜色设置；
4. 多边形边数输入弹窗；
5. 清空画布；
6. 选择工具与部分图元高亮显示；
7. 画布区域裁剪显示与统一鼠标位置映射。

3.2 界面与测试示例

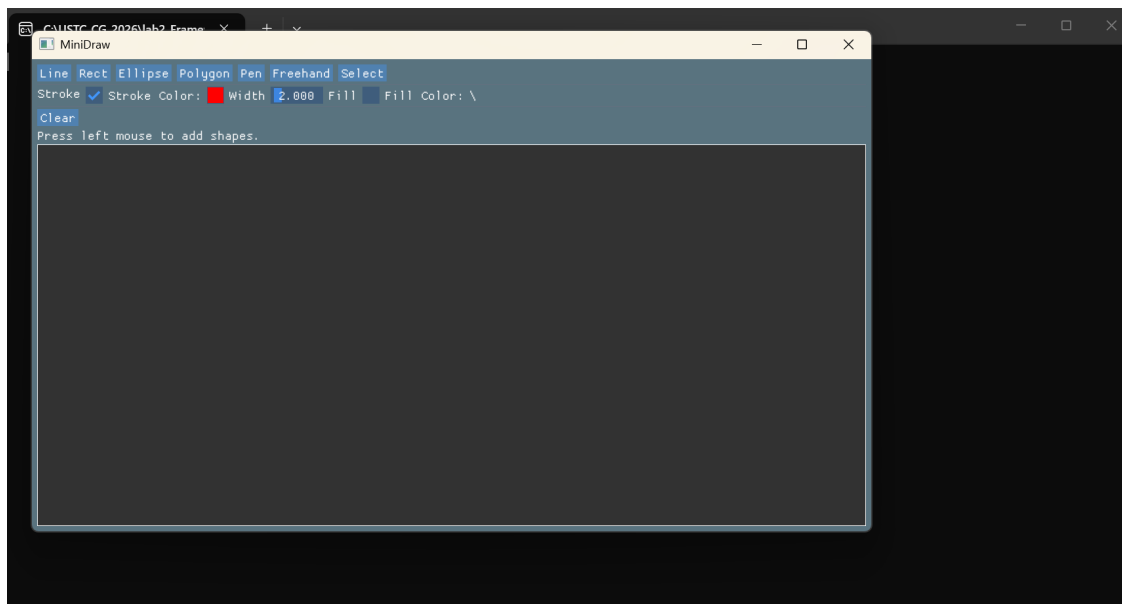
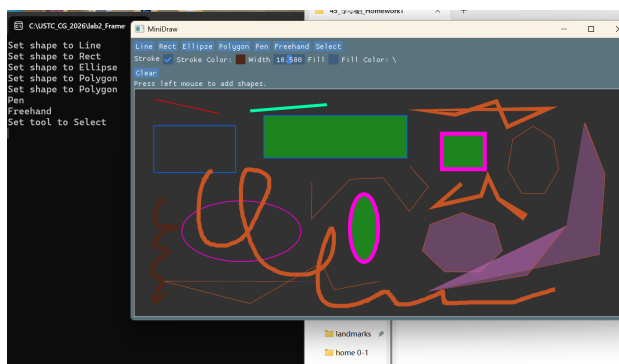


图 1: MiniDraw 程序界面截图



(a) 演示图

3.3 类图说明

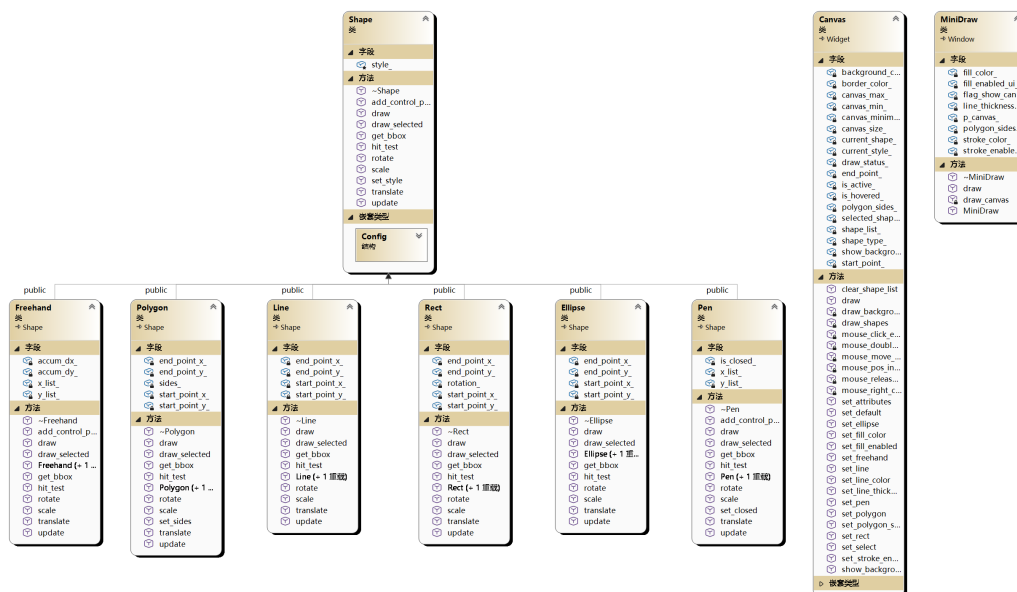


图 3: 主要类图

从类图上看，Shape 构成了图元层的抽象基类，Line、Rect、Ellipse、Polygon、Pen、Freehand 作为派生类分别实现各自几何行为；Canvas 负责调度与管理图元对象；MiniDraw 负责界面入口与控件组织。这一结构符合课程作业“继承 + 多态”的设计目标。

4 结论

本实验基于 Framework2D 与 ImGui，完成了一个小型矢量绘图工具 MiniDraw 的原型实现。程序使用 Shape 基类统一抽象图元接口，并通过多态管理直线、矩形、椭圆、规则多边形、钢笔折线与自由划线等不同对象；同时基于 Canvas 建立了统一的鼠标事件分发与交互状态管理机制，实现了基础绘制、样式设置和部分图元选择高亮功能。